

Competitive Programming

Section 15 Editorial: BFS And Unweighted Shortest Paths

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	4
2	BFS reminder	5
3	A. Connected Groups	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Walk-through	7
3.5	Algorithm	7
3.6	Correctness proof	8
3.7	Complexity	8
3.8	C++ implementation notes	8
3.9	Python implementation notes	8
3.10	Full C++ solution	8
3.11	Full Python solution	10
3.12	Common wrong answers	11
3.13	Tests that matter	11
4	B. Unweighted Routes	12
4.1	Restatement	12
4.2	Constraints and traps	12
4.3	Observations	12
4.4	Walk-through	12
4.5	Algorithm	13
4.6	Correctness proof	13
4.7	Complexity	14
4.8	Full C++ solution	14
4.9	Full Python solution	15
4.10	Common wrong answers	16
4.11	Tests that matter	16
5	C. Grid Rescue Path	17
5.1	Restatement	17
5.2	Constraints and traps	17
5.3	Observations	17
5.4	Walk-through	18
5.5	Algorithm	18
5.6	Correctness proof	19
5.7	Complexity	19
5.8	Full C++ solution	20
5.9	Full Python solution	21
5.10	Common wrong answers	22
5.11	Tests that matter	22

6	Stress testing	23
7	What to carry forward	24

1 Editorial overview

All three problems use breadth-first search, but in different roles:

Problem	BFS role
A. Connected Groups	visit one whole connected component
B. Unweighted Routes	compute shortest distances from one source
C. Grid Rescue Path	compute shortest path and reconstruct moves

The shared invariant is:

The queue processes vertices in nondecreasing distance from the BFS start.

2 BFS reminder

BFS uses a first-in, first-out queue.

Core template:

```
mark start discovered
push start

while queue is not empty:
    u = pop front
    for each neighbor v:
        if v is not discovered:
            mark v discovered
            record information about v
            push v
```

For components, “record information” means increasing a size counter. For distances, it means $\text{dist}[v] = \text{dist}[u] + 1$. For path output, it also means recording the parent used to reach v .

3 A. Connected Groups

3.1 Restatement

For each undirected graph, find every connected component and output the component sizes in increasing order.

Two vertices are in the same connected component if there is some path between them.

3.2 Constraints and traps

The total graph size is large enough that the intended complexity is linear:

```
0(n + m) per test case
```

Traps:

- isolated vertices are components of size 1;
- repeated edges should not break the algorithm;
- the graph is undirected, so every input edge must be stored both ways;
- recursive DFS can overflow on a long chain, so use iterative BFS here.

3.3 Observations

If we start BFS from an unseen vertex s , BFS visits exactly the vertices in s 's connected component:

- every visited vertex is reachable from s because BFS follows graph edges;
- every vertex reachable from s will eventually be discovered because BFS expands all outgoing edges of every discovered vertex.

So the algorithm is:

```
for every vertex:
    if it is unseen:
        run BFS from it
        count how many vertices are reached
```

3.4 Walk-through

Graph:

```
1 -- 2 -- 3      4 -- 5      6
```

Loop:

```
start 1 -> BFS sees 1,2,3 -> size 3
start 2 -> already seen
start 3 -> already seen
start 4 -> BFS sees 4,5 -> size 2
start 5 -> already seen
start 6 -> BFS sees 6 -> size 1
```

After sorting, output:

```
1 2 3
```

3.5 Algorithm

```
read n,m
adj = empty list for each vertex
for each edge u,v:
    add v to adj[u]
    add u to adj[v]

seen = false for every vertex
sizes = []

for start in 0..n-1:
    if seen[start]:
        continue
    seen[start] = true
    queue = [start]
    size = 0
    while queue not empty:
        u = pop front
        size += 1
        for v in adj[u]:
            if not seen[v]:
                seen[v] = true
                push v
        sizes.append(size)

sort sizes
print count and sizes
```

3.6 Correctness proof

Lemma 1. A BFS started from s visits only vertices in the connected component of s .

Proof. BFS starts at s . Every new vertex is added by following an edge from a vertex already reached from s . Therefore every visited vertex has a path from s . $\square$

Lemma 2. A BFS started from s visits every vertex in the connected component of s .

Proof. Let x be any vertex reachable from s , and consider a shortest path from s to x : $s = p_0, p_1, \dots, p_k = x$. BFS starts with p_0 . Whenever p_i is visited, BFS scans its neighbors, so if p_{i+1} has not already been visited, it is discovered. By induction along the path, x is visited. $\square$

Lemma 3. The outer loop starts exactly one BFS per connected component.

Proof. When the loop first reaches a vertex of a component, that vertex is unseen and starts a BFS. By Lemma 2, this BFS marks the entire component seen. All later vertices in the same component are skipped. $\square$

Theorem. The algorithm outputs exactly the sizes of all connected components.

Proof. By Lemma 3, each component contributes one BFS. By Lemmas 1 and 2, that BFS visits exactly the vertices of that component, so its counter is the component size. Sorting only changes output order, not the multiset of sizes. $\square$

3.7 Complexity

Each vertex enters the queue once. Each undirected edge appears in two adjacency lists and is scanned at most twice.

```
Time:  $O(n + m)$   
Memory:  $O(n + m)$ 
```

3.8 C++ implementation notes

Use `vector<vector<int>> adj(n)` and `queue<int>`. Use `vector<char>` for visited flags.

3.9 Python implementation notes

Use `collections.deque`. Read all integers with `sys.stdin.buffer.read()` for large input.

3.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        int n, m;
        cin >> n >> m;
        vector<vector<int>> adj(n);
        for (int i = 0; i < m; i++) {
            int u, v;
            cin >> u >> v;
            --u;
            --v;
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<char> seen(n, false);
        vector<int> sizes;
        for (int start = 0; start < n; start++) {
            if (seen[start]) continue;
            seen[start] = true;
            queue<int> q;
            q.push(start);
            int size = 0;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                size++;
                for (int v : adj[u]) {
                    if (!seen[v]) {
                        seen[v] = true;
                        q.push(v);
                    }
                }
            }
            sizes.push_back(size);
        }

        sort(sizes.begin(), sizes.end());
        cout << sizes.size() << '\n';
        for (int i = 0; i < (int)sizes.size(); i++) {
            if (i) cout << ' ';
            cout << sizes[i];
        }
        cout << '\n';
    }
}

```

```

    }
    return 0;
}

```

3.11 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    t = next(it)
    out = []
    for _ in range(t):
        n = next(it)
        m = next(it)
        adj = [[] for _ in range(n)]
        for _ in range(m):
            u = next(it) - 1
            v = next(it) - 1
            adj[u].append(v)
            adj[v].append(u)

        seen = [False] * n
        sizes = []
        for start in range(n):
            if seen[start]:
                continue
            seen[start] = True
            q = deque([start])
            size = 0
            while q:
                u = q.popleft()
                size += 1
                for v in adj[u]:
                    if not seen[v]:
                        seen[v] = True
                        q.append(v)
            sizes.append(size)

        sizes.sort()
        out.append(str(len(sizes)))
        out.append(" ".join(map(str, sizes)))
    sys.stdout.write("\n".join(out))

```

```
if __name__ == "__main__":  
    main()
```

3.12 Common wrong answers

- Storing only `adj[u].append(v)` for an undirected graph.
- Forgetting isolated vertices.
- Counting edges instead of vertices.
- Sorting vertices inside each component instead of sorting component sizes.

3.13 Tests that matter

Use tests with:

- no edges;
- one long chain;
- repeated edges;
- several components with equal sizes;
- a single vertex.

4 B. Unweighted Routes

4.1 Restatement

Given a directed graph and a source s , answer many queries of the form:

What is the minimum number of edges needed to reach x from s ?

Print -1 if x cannot be reached.

4.2 Constraints and traps

The source is the same for every query. Therefore we should not run BFS for each query.

Traps:

- edges are directed;
- the source has distance 0 from itself;
- self-loops and repeated edges do not need special handling;
- unreachable vertices must remain -1 .

4.3 Observations

All edges have equal cost: one edge is one move. BFS from s computes the minimum number of edges from s to every reachable vertex in one traversal.

After that, each query is just an array lookup.

4.4 Walk-through

Graph:

```
1 -> 2 -> 4 -> 5
|      |
v      v
3 -> 6
```

From 1:

```

dist[1] = 0
dist[2] = 1
dist[3] = 1
dist[4] = 2
dist[6] = 2
dist[5] = 3

```

Any vertex never discovered gets distance -1.

4.5 Algorithm

```

read graph
dist = [-1] * n
dist[s] = 0
queue = [s]

while queue not empty:
    u = pop front
    for v in adj[u]:
        if dist[v] == -1:
            dist[v] = dist[u] + 1
            push v

for each query x:
    print dist[x]

```

4.6 Correctness proof

Lemma 1. When BFS assigns $\text{dist}[v]$, there is a path from s to v of length $\text{dist}[v]$.

Proof. The source has a path of length 0. Every other assignment is made from an edge $u \rightarrow v$ after u already has a path of length $\text{dist}[u]$; adding that edge gives a path of length $\text{dist}[u] + 1$. $\square$

Lemma 2. BFS never assigns a distance larger than the true shortest distance.

Proof. This follows from Lemma 1: assigned distances correspond to real paths. $\square$

Lemma 3. BFS never assigns a distance smaller than the true shortest distance.

Proof. Suppose a vertex v has a shortest path of length d : $s = p_0, p_1, \dots, p_d = v$. BFS processes vertices by increasing assigned distance. By induction, p_{d-1} is discovered with distance at most $d-1$, so when it is processed, v is discovered with distance at most d if it was not already discovered. Therefore the first possible discovery of v happens no later than layer d , and cannot be below the true shortest distance by Lemma 2. $\square$

Theorem. For every query target x , the algorithm prints the shortest distance from s to x , or -1 if unreachable.

Proof. If x is reachable, Lemmas 2 and 3 show that BFS assigns exactly the shortest distance. If x is unreachable, no path can lead BFS to it, so its distance remains -1 . Each query prints this computed value. $\square$

4.7 Complexity

The BFS is run once.

```
Time:  $O(n + m + q)$   
Memory:  $O(n + m)$ 
```

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s, q_count;
    if (!(cin >> n >> m >> s >> q_count)) return 0;
    --s;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
    }
    vector<int> queries(q_count);
    for (int &x : queries) {
        cin >> x;
        --x;
    }

    vector<int> dist(n, -1);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
            }
        }
    }
}
```

```

        q.push(v);
    }
}

for (int i = 0; i < q_count; i++) {
    if (i) cout << ' ';
    cout << dist[queries[i]];
}
cout << '\n';
return 0;
}

```

4.9 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    s = next(it) - 1
    q_count = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
    queries = [next(it) - 1 for _ in range(q_count)]

    dist = [-1] * n
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)

    print(" ".join(str(dist[x]) for x in queries))

```

```
if __name__ == "__main__":  
    main()
```

4.10 Common wrong answers

- Running BFS once per query.
- Treating directed edges as undirected.
- Initializing the source distance to 1.
- Using 0 as the undiscovered marker, which conflicts with the source distance.

4.11 Tests that matter

Use tests with:

- unreachable vertices;
- target equal to source;
- directed edge that cannot be traversed backward;
- self-loop;
- repeated edges.

5 C. Grid Rescue Path

5.1 Restatement

Find a shortest path from S to G in a grid with walls. Moves are D, L, R, and U.

If multiple shortest paths exist, output the lexicographically smallest move string under:

```
D < L < R < U
```

If no path exists, print NO.

5.2 Constraints and traps

The grid can contain up to 10^6 cells, so the intended solution is linear in the number of cells.

Traps:

- check bounds before reading `grid[nr][nc]`;
- walls cannot be entered;
- the path must be reconstructed in forward order;
- neighbor order matters because of the tie-breaking rule;
- do not replace an existing parent when the cell is reached again.

5.3 Observations

A grid is a graph:

```
cell = vertex  
legal one-step move = edge
```

Every move has cost 1, so BFS gives shortest distances.

To output the path, store for each discovered cell:

```
parent row  
parent column  
move character used from parent to this cell
```

To get the required lexicographically smallest shortest path, scan neighbors in this order:

```
D, L, R, U
```

Because BFS processes all shorter paths before longer paths, and because we keep the first parent assigned to each cell, this order fixes the tie between same-length paths.

5.4 Walk-through

Grid:

```
#####  
#S...G#  
#.###.#  
#.....#  
#####
```

From S, BFS discovers cells by distance:

```
distance 0: S  
distance 1: right cell, lower cell  
distance 2: ...
```

The goal is first reached by:

```
R R R R
```

So the output length is 4 and the path is RRRR.

5.5 Algorithm

```
find S and G  
dist = -1 for every cell  
parent = empty for every cell  
dist[S] = 0  
queue = [S]  
  
while queue not empty:  
    r,c = pop front  
    for moves in order D,L,R,U:  
        nr,nc = neighbor  
        if outside grid: continue  
        if wall: continue  
        if already discovered: continue  
        dist[nr][nc] = dist[r][c] + 1  
        parent[nr][nc] = (r,c,move)  
        push (nr,nc)
```

```

if G undiscovered:
    print NO
else:
    walk backward from G using parent
    reverse collected moves
    print YES, length, path

```

5.6 Correctness proof

Lemma 1. Every discovered grid cell has a valid path from S whose move string is obtained by following parent pointers.

Proof. S is discovered with the empty path. Every other cell is discovered from an already discovered adjacent non-wall cell using one legal move. Recording the parent preserves that legal predecessor. Induction over discovery order proves the claim. $\square$

Lemma 2. The first time BFS discovers a cell, the recorded distance is the shortest possible distance from S .

Proof. This is the standard BFS layer invariant. The queue processes all cells at distance k before cells at distance $k+1$. Therefore a cell cannot first be discovered through a longer path while an undiscovered shorter path to it exists. $\square$

Lemma 3. Among all shortest paths to a cell, the first parent assignment obtained with neighbor order D,L,R,U gives the lexicographically smallest move string under $D < L < R < U$.

Proof. BFS processes path prefixes in lexicographic order within the same distance layer because each frontier cell was itself first discovered by the smallest prefix leading to it, and its outgoing moves are considered in lexicographic order. A cell's first discovery among shortest paths is therefore through the smallest shortest prefix plus move that reaches it. The algorithm does not overwrite parents, so this smallest shortest path is kept. $\square$

Theorem. If the algorithm prints YES, it prints the lexicographically smallest shortest path from S to G ; if it prints NO, no such path exists.

Proof. If G is undiscovered, BFS has exhausted all reachable cells, so no path exists. If G is discovered, Lemma 2 gives shortest length and Lemma 3 gives the required tie-breaking. Lemma 1 shows the reconstructed string is a valid path. $\square$

5.7 Complexity

Each cell is pushed at most once, and each push checks four neighbors.

```

Time:  $O(n*m)$ 
Memory:  $O(n*m)$ 

```

5.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Parent {
    int r = -1;
    int c = -1;
    char move = '?';
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<string> grid(n);
    pair<int, int> start{-1, -1}, goal{-1, -1};
    for (int r = 0; r < n; r++) {
        cin >> grid[r];
        for (int c = 0; c < m; c++) {
            if (grid[r][c] == 'S') start = {r, c};
            if (grid[r][c] == 'G') goal = {r, c};
        }
    }

    const vector<int> dr = {1, 0, 0, -1};
    const vector<int> dc = {0, -1, 1, 0};
    const vector<char> mv = {'D', 'L', 'R', 'U'};

    vector<vector<int>> dist(n, vector<int>(m, -1));
    vector<vector<Parent>> parent(n, vector<Parent>(m));
    queue<pair<int, int>> q;
    dist[start.first][start.second] = 0;
    q.push(start);
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i];
            int nc = c + dc[i];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            if (grid[nr][nc] == '#' || dist[nr][nc] != -1) continue;
            dist[nr][nc] = dist[r][c] + 1;
            parent[nr][nc] = {r, c, mv[i]};
            q.push({nr, nc});
        }
    }
}
```

```

auto [gr, gc] = goal;
if (dist[gr][gc] == -1) {
    cout << "NO\n";
    return 0;
}

string path;
for (pair<int, int> cur = goal; cur != start;) {
    Parent p = parent[cur.first][cur.second];
    path.push_back(p.move);
    cur = {p.r, p.c};
}
reverse(path.begin(), path.end());
cout << "YES\n" << path.size() << '\n' << path << '\n';
return 0;
}

```

5.9 Full Python solution

```

from collections import deque
import sys

DIRS = [(1, 0, "D"), (0, -1, "L"), (0, 1, "R"), (-1, 0, "U")]

def main() -> None:
    input = sys.stdin.readline
    n, m = map(int, input().split())
    grid = [list(input().strip()) for _ in range(n)]
    start = goal = (-1, -1)
    for r in range(n):
        for c in range(m):
            if grid[r][c] == "S":
                start = (r, c)
            elif grid[r][c] == "G":
                goal = (r, c)

    dist = [[-1] * m for _ in range(n)]
    parent = [[None] * m for _ in range(n)]
    sr, sc = start
    gr, gc = goal
    dist[sr][sc] = 0
    q = deque([start])
    while q:
        r, c = q.popleft()
        for dr, dc, move in DIRS:
            nr, nc = r + dr, c + dc

```

```

        if not (0 <= nr < n and 0 <= nc < m):
            continue
        if grid[nr][nc] == "#" or dist[nr][nc] != -1:
            continue
        dist[nr][nc] = dist[r][c] + 1
        parent[nr][nc] = (r, c, move)
        q.append((nr, nc))

    if dist[gr][gc] == -1:
        print("NO")
        return

    path = []
    r, c = goal
    while (r, c) != start:
        pr, pc, move = parent[r][c]
        path.append(move)
        r, c = pr, pc
    path.reverse()
    print("YES")
    print(len(path))
    print("".join(path))

if __name__ == "__main__":
    main()

```

5.10 Common wrong answers

- Reconstructing from S forward without stored children.
- Forgetting to reverse the backward path.
- Using neighbor order U,D,L,R, which breaks the required tie-breaking.
- Reassigning a parent when another shortest path reaches the same cell.
- Printing YES 0 for unreachable instead of NO.

5.11 Tests that matter

Use tests with:

- direct straight path;
- unreachable goal;
- multiple shortest paths requiring tie-breaking;
- walls along the border and open borders;
- large open grid.

6 Stress testing

For Section 15, random testing is very effective.

Problem A can generate small random undirected graphs and compare BFS component sizes against a second implementation such as DSU or Floyd reachability on tiny graphs.

Problem B can generate small directed graphs and compare BFS distances against all-pairs Floyd-Warshall on tiny graphs, treating each edge as weight 1.

Problem C can generate random grids and compare:

```
reported length == BFS distance to G
reported path starts at S
reported path never enters a wall
reported path ends at G
```

The local generators for this section compute expected answers directly with the same invariants explained here, then run both C++ and Python reference solutions against fixed and random cases.

7 What to carry forward

BFS is the default tool for:

```
unweighted shortest path  
reachability  
connected components  
grid shortest path  
state-space shortest path
```

Before using BFS, always check the cost of a move. If all moves cost the same, BFS is probably right. If costs differ, wait for weighted shortest paths.